
Game Reunion

Autor: Rodrigo Esparís Álvarez

Ciclo / Curso: 2º DAM

Fecha: 29/06/2026

Tecnologías principales: Dart · Flutter · Java · MySQL

ÍNDICE

- 1. Resumen**
- 2. Motivación e introducción**
- 3. Metodología utilizada**
 - a. Recursos Software y Hardware
 - b. Planificación
- 4. Tecnologías y herramientas**
 - a. Listado breve
- 5. Análisis**
 - a. Requisitos funcionales y no funcionales
 - b. Modelo de datos y justificación de DB
 - c. Casos de uso
- 6. Diseño**
 - a. Diseño general
 - b. Diagrama de clases
 - c. Interfaz
 - d. Decisiones de diseño
- 7. Implementación**
 - a. Estructura del proyecto
 - b. Componentes principales
 - c. Dificultades encontradas
- 8. Multiusuario concurrente**
 - a. Explicación y pruebas
- 9. Despliegue e instalación**
- 10. Pruebas**
 - a. Plan de pruebas
 - b. Casos de prueba
- 11. Resultados, conclusiones y vías futuras**
 - a. Objetivos alcanzados
 - b. Conclusiones
 - c. Vías futuras
- 12. Glosario**
- 13. Bibliografía / Webgrafia**

Resumen

El proyecto es una app de móvil para usuarios que les gusten los videojuegos multijugador, sirve como intermediario para que la gente encuentre otras personas con las que jugar. Esto evita jugar cada partida con gente distinta y permite conocer gente nueva con la que jugar a diario. Esto se hace mediante las funciones de crear y unirse a grupos que tengan el mismo objetivo y juego, en el grupo se pueden mandar mensajes para introducirse un poco o empezar de forma más tranquila con la gente nueva, o unirse a una llamada con ellos por la que se puede comunicar por voz sin necesidad de usar el chat del juego que puede llegar a ser con más personas

1- Motivación e introducción

El proyecto surge de la situación de mucha gente que juega sola en sus partidas ya que no tiene amigos que jueguen a ese juego con ellos, lo que hace que se pierda una parte de la experiencia. Con esta aplicación los usuarios podrán tanto jugar con gente temporalmente durante varias horas para que su experiencia sea más entretenida, o conocer a gente nueva con la que puede que jueguen durante mucho tiempo y se hagan amigos.

El problema que había antes es que la gente cuando intentaba buscar este tipo de grupos cuando entraba en partida nadie se comunicaba o ya tenían su propio grupo de amigos cerrado, o por otro lado servidores de Discord que intentan tener esta funcionalidad acaban desapareciendo ya que estos servidores suelen usarse para otras cosas, y esta funcionalidad queda atrás. Con esta aplicación siendo únicamente dedicada a este entorno siempre se intentará actualizar con frecuencia añadiendo funciones o mejoras en torno al usuario, o mejoras para cada juego por separado (Funciones de hablar por turnos en juegos como "Among Us").

El objetivo general de la aplicación es ofrecer a los jugadores de videojuegos multijugador un espacio dedicado exclusivamente a encontrar compañeros de juego y comunicarse con ellos de forma cómoda y continua, eliminando la dependencia de herramientas externas como los chats de los propios juegos o servidores de Discord no especializados.

Para este objetivo, se plantean los siguientes objetivos específicos. Primero se implementa un sistema de registro e inicio de sesión seguro que garantiza que cada usuario tenga su propia cuenta con acceso desde sus credenciales.

Esto se consigue mediante un sistema HASH, que codifica las contraseñas al almacenarlas, de esta forma, usando la contraseña podremos averiguar su código HASH, pero con el código HASH, no podremos averiguar la contraseña.

El HASH se almacena en la base de datos, haciendo que al poner la contraseña, esta, se transforme en su código HASH, el backend verifica que son idénticos, una vez verificado, permite el acceso al usuario.

Por otro lado, al crear la cuenta, se tiene que verificar mediante correo electrónico, usando el código que se mandará, para evitar que la gente cree cuentas con correos ajenos.

En segundo lugar, se desarrolla un sistema de creación y gestión de grupos, donde se indica información del juego, cantidad de gente máxima que buscan, o qué personas ya están en el grupo, permitiendo a otros usuarios unirse a ellos o crear los grupos dependiendo de sus preferencias.

En tercer lugar, se integra un chat de mensajes de texto en tiempo real dentro de cada uno de los grupos creados, útil para miembros que se incorporan recientemente o que prefieren no usar el chat de voz en el primer momento. Por otro lado se implementan llamadas de voz grupales siempre activas, para que cualquier usuario pueda comunicarse mientras juega. En quinto lugar, existe soporte de varios usuarios simultáneos, asegurando consistencia de datos y estabilidad en las comunicaciones. Por último, se diseña una interfaz intuitiva y relacionada con la temática de videojuegos, para que resulte familiar y accesible.

2 - Metodología utilizada

La metodología utilizada fue una metodología incremental, donde en cada fase del proyecto se establecen bases que complementaban a la fase anterior, añadiendo nuevas funcionalidades de forma progresiva.

En la primera fase se desarrolló el backend, implementando los métodos principales y verificando que cada uno realizará de forma correcta las consultas necesarias. Una vez validada la lógica, se integró una base de datos para comprobar la persistencia de los datos.

En la segunda fase se creó un sistema de autenticación, haciendo que cada usuario tuviera sus propios datos individuales, accesibles únicamente mediante sus credenciales.

En la tercera fase, con el producto mínimo viable ya implementado, se fue construyendo una interfaz visual conectada a todos los métodos desarrollados.

Finalmente, se fueron añadiendo nuevas funcionalidades sobre esta interfaz, incorporando nuevas pestañas y botones, y comprobando en cada paso que no alteraban los flujos ya funcionales.

Recursos(Hardware)

Para el desarrollo se utilizaron dos equipos, un ordenador sobremesa y un portátil, ambos Windows. No se requirió un servidor externo ya que se trabajaba en local durante el desarrollo para la base de datos y el backend. En etapas más avanzadas la base de datos fue migrada a la nube.

No es necesario tener los dos equipos funcionando simultáneamente, ya que se puede realizar el trabajo y pruebas en 1 solo dispositivo, simulando virtualmente los móviles, sin embargo, tener dos ordenadores facilita mucho la simulación de multiusuario.

Para el uso de la aplicación se requiere un dispositivo Android y conexión a internet junto con la aplicación descargada ya sea en su formato producción o desarrollo

En caso de querer usar un dispositivo virtual, se recomienda tener un equipo bastante potente, ya que la herramienta de Android Studio utiliza muchos recursos, y un equipo con pocos requisitos podría tardar mucho en iniciar la aplicación o ir muy lento en la navegación entre pantallas o en las consultas.

Recursos(Software)

Para el desarrollo del proyecto se utilizaron dos lenguajes principales, Dart para el frontend y Java para el backend. Como frameworks se emplearon Flutter para construir la interfaz móvil y Spring Boot para desarrollar la API REST.

La base de datos utilizada fue MySQL, gestionada localmente durante el desarrollo y migrada a la nube mediante Aiven en fases más avanzadas. Para el desarrollo del frontend se utilizó Android Studio como entorno principal, mientras que el backend se desarrolló en Visual Studio Code.

Las pruebas y validación de los endpoints de la API se realizaron con Postman y terminal de windows, y el control de versiones se gestionó mediante Git y GitHub.

Planificación

Fase 1 - Desarrollo de backend y métodos principales, para funcionalidades básicas y desarrollo de diagramas y guías - Últimas semanas de marzo

Fase 2 - Integración de base de datos, métodos CRUD, para acceso a la base de datos y verificación de información - Principios de Abril

Fase 3 - Sistema de autenticación de usuarios, evitar usuarios duplicados, y multiusuario - Principios de Abril

Fase 4 - Conexión del frontend con el backend para el acceso a las llamadas, y mostrar información en el móvil - Mediados de abril

Fase 5 - Trabajar en verificaciones, gestión de errores, modificaciones para mejoras - Finales de abril y principios de Mayo

Fase 6 - Correcciones finales y redacción de README, documentos y presentación

3 - Tecnologías y herramientas

Lenguajes: Dart - 3.11.4 · Java - 21.0.10

Frameworks: Flutter (frontend móvil) - 3.41.6 · Spring Boot (backend/API REST) - 3.2.5

Base de datos: MySQL - 8.0.47, alojada en la nube mediante Aiven

Comunicación en tiempo real: WebSocket, utilizado tanto para la mensajería instantánea como para las llamadas de voz grupales

IDE: Android Studio (frontend) - Panda 3 | 2025.3.3 · Visual Studio Code (backend) - 1.123

Pruebas de API: Postman - Terminal de Windows

Control de versiones: Git · GitHub - 2.54.0

Despliegue de aplicación: Docker Desktop - 4.76.0

4 - Análisis

Requisitos

- **Funcionales**
 - Permite a los usuarios registrarse con un correo, un nombre de usuario, su nombre y una contraseña
 - Permite iniciar sesión con cuentas ya creadas usando su nombre de usuario y contraseña
 - Permite unirse a grupos existentes
 - Permite crear grupos para que otras jugadores se unan
 - Puedes salir del grupo para unirte a uno nuevo
 - Se pueden enviar mensajes entre usuarios en un grupo
 - Puedes unirte a una llamada grupal (siempre activa) para hablar con otros usuarios
- **No funcionales**
 - El servicio de mensajes está siempre disponible
 - Tiene una interfaz intuitiva y fácil de entender, que se relaciona con la temática de la aplicación
 - La aplicación solo permite estar en un grupo a la vez
 - Las llamadas siempre están abiertas mientras exista alguien en ella, aunque esté solo

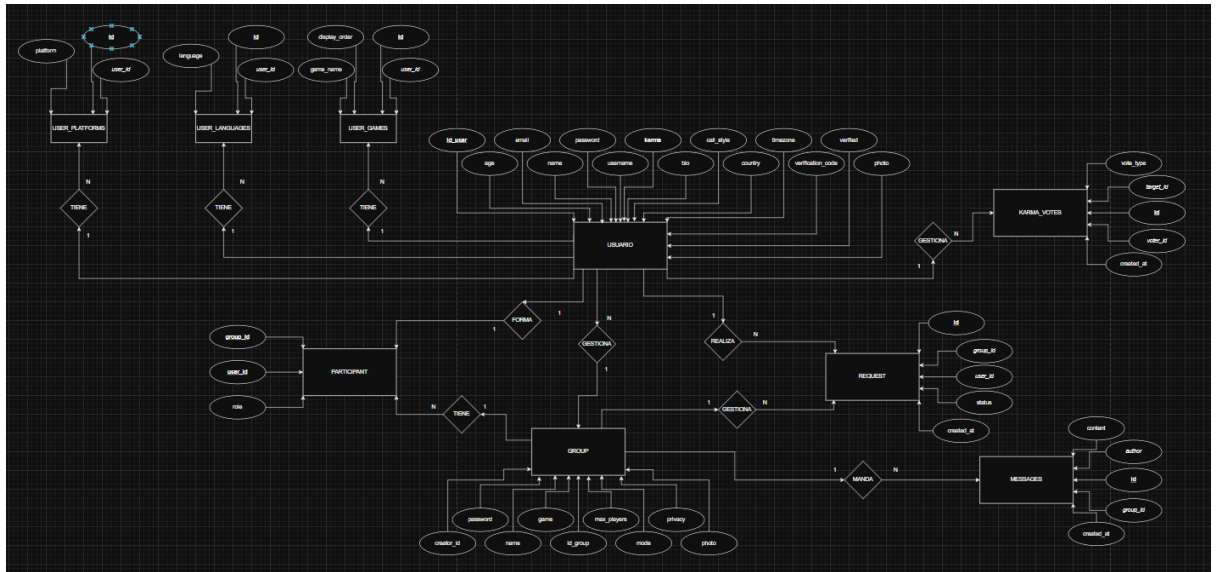
Modelo de datos

La aplicación usa un sistema de base de datos relacional para tener la información de forma estructurada, garantizando así que los datos sean reales y no den errores, es decir, que tengan integridad, y presentando así facilidades de relación entre las entidades.

Entidades / Tablas principales

- **Usuario**
 - Guarda las diferentes cuentas que se registran
 - Tiene campos de datos personales como el nombre, la contraseña, o el correo
 - Otros campos que contiene son, puntos de karma entre otros valores
- **Grupos**
 - Aquí se registran los diferentes grupos que crean los usuarios, estos son más dinámicos ya que cambian constantemente cuando se crean, borran, o se quedan vacíos.
 - Contiene campos como su nombre, su máximo de miembros, el juego al que están jugando
- **User_Games/Languages/Platforms**
 - Estas tres tablas se encargan de almacenar datos de los usuarios
 - Se almacenan en tablas aparte debido a que los usuarios pueden tener varios juegos, varios idiomas, y varias plataformas en las que jugar. Esto hace que meterlos en la tabla de usuarios crear múltiples instancias de un mismo usuario
- **Karma votes**
 - Los votos se guardan en cada usuario, esta tabla es la encargada de registrar quien realizó el votó, a quien se lo dió, y si fue positivo y negativo
 - Esto sirve para poder tener en cuenta un límite de votos diarios a cada usuario, y tener una cantidad limitada de votos positivos o negativos de un usuario concreto a un mismo usuario repetidamente
- **Messages**
 - Una tabla para almacenar los mensajes de los grupos
 - Al almacenarlos en la tabla, puedes tener un registro de los mensajes, para poner penalizaciones o restricciones en caso de un abuso, o palabras indebidas.
 - Por otro lado también sirve para que usuarios que se unen a un grupo después de que se manden los mensajes, puedan leer los mensajes de antes de haberse unido al grupo
- **Participants**
 - Registra en qué grupo está un usuario, y qué usuarios están en un grupo
 - Permite que un usuario no esté en dos grupos simultáneamente
 - También sirve para saber qué rol tiene cada usuario en el grupo, ya que cuando se cambie de grupo, su rol(en caso de ser líder o admin), se perderá

Diagrama de ER



Justificación:

Se seleccionó esta base de datos debido a que la estructura de la base de datos debería ser constante, y no se alteraría en muchos casos, haciéndola mejor que la documental, por otro lado así evitamos duplicidad.

Concretando más sobre la base de datos, se seleccionó esta debido a su simplicidad, ya que las consultas que se realizan en la aplicación no tienen necesidad de ser complejas, y por último, su compatibilidad con muchas herramientas, ya que al usar múltiples lenguajes, frameworks y bases de datos en nube, esto nos facilitará trabajo.

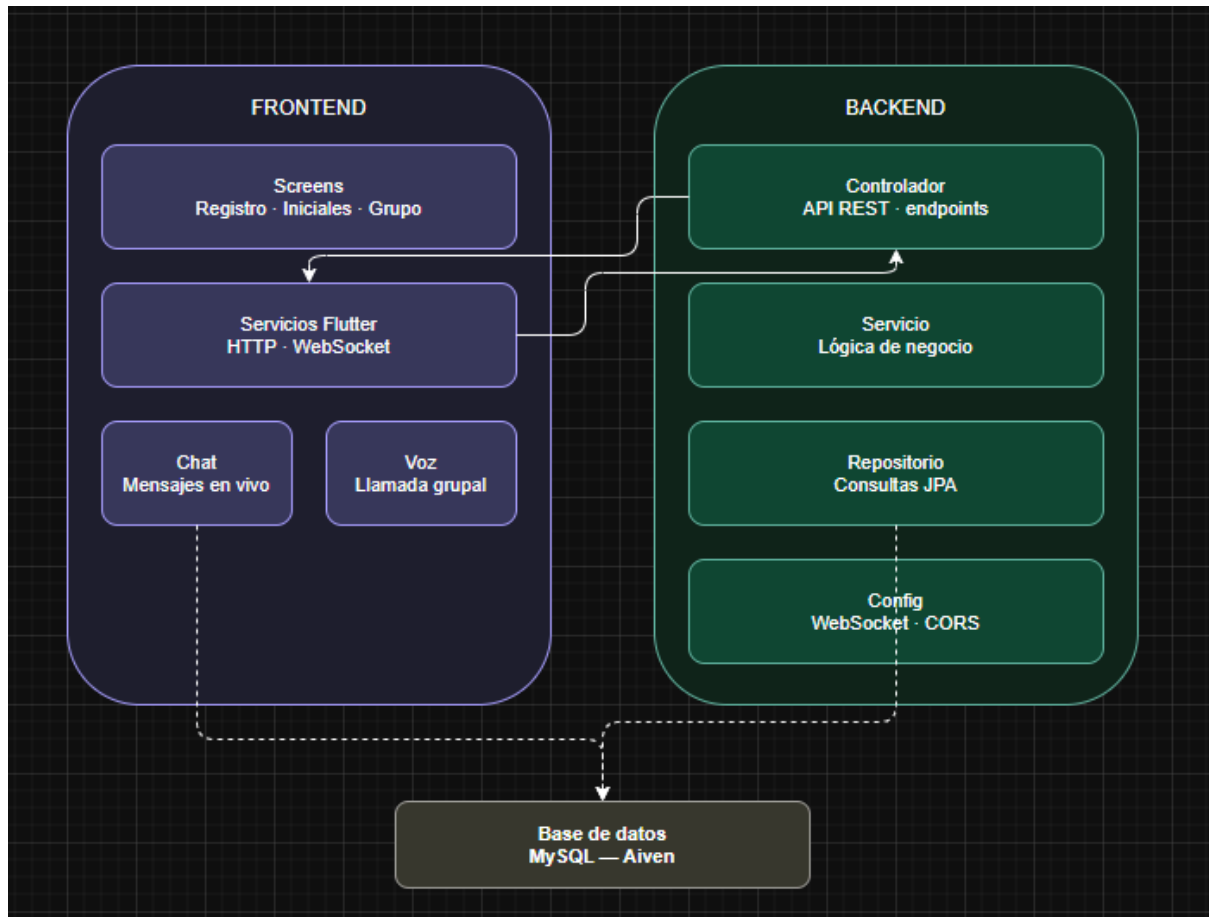
Casos de uso

1. Usuario no registrado
 - a. Registrarse en la aplicación usando correo, nombre, nombre de usuario y contraseña
 - b. Iniciar sesión con un nombre de usuario y contraseña ya existentes
2. Miembro
 - a. Unirse a un grupo existente enviando una solicitud
 - b. Abandonar el grupo en el que se encuentra
 - c. Enviar y recibir mensajes de texto en tiempo real dentro del grupo
 - d. Unirse y abandonar la llamada de voz
 - e. Crear un grupo configurando nombre, juego, y número máximo de jugadores
3. Admin (Además de lo anterior)
 - a. Aceptar o rechazar solicitudes de entrada al grupo
 - b. Expulsar miembros del grupo, excepto a otros administradores o al líder
 - c. Ascender el rol de un miembro
4. Líder (Además de lo anterior)
 - a. Gestionar un grupo sin restricciones
 - b. Cerrar o eliminar un grupo
 - c. Pasar su liderazgo a otro usuario

(Revisar Anexo 1 para más información)

5 - Diseño

Diseño General



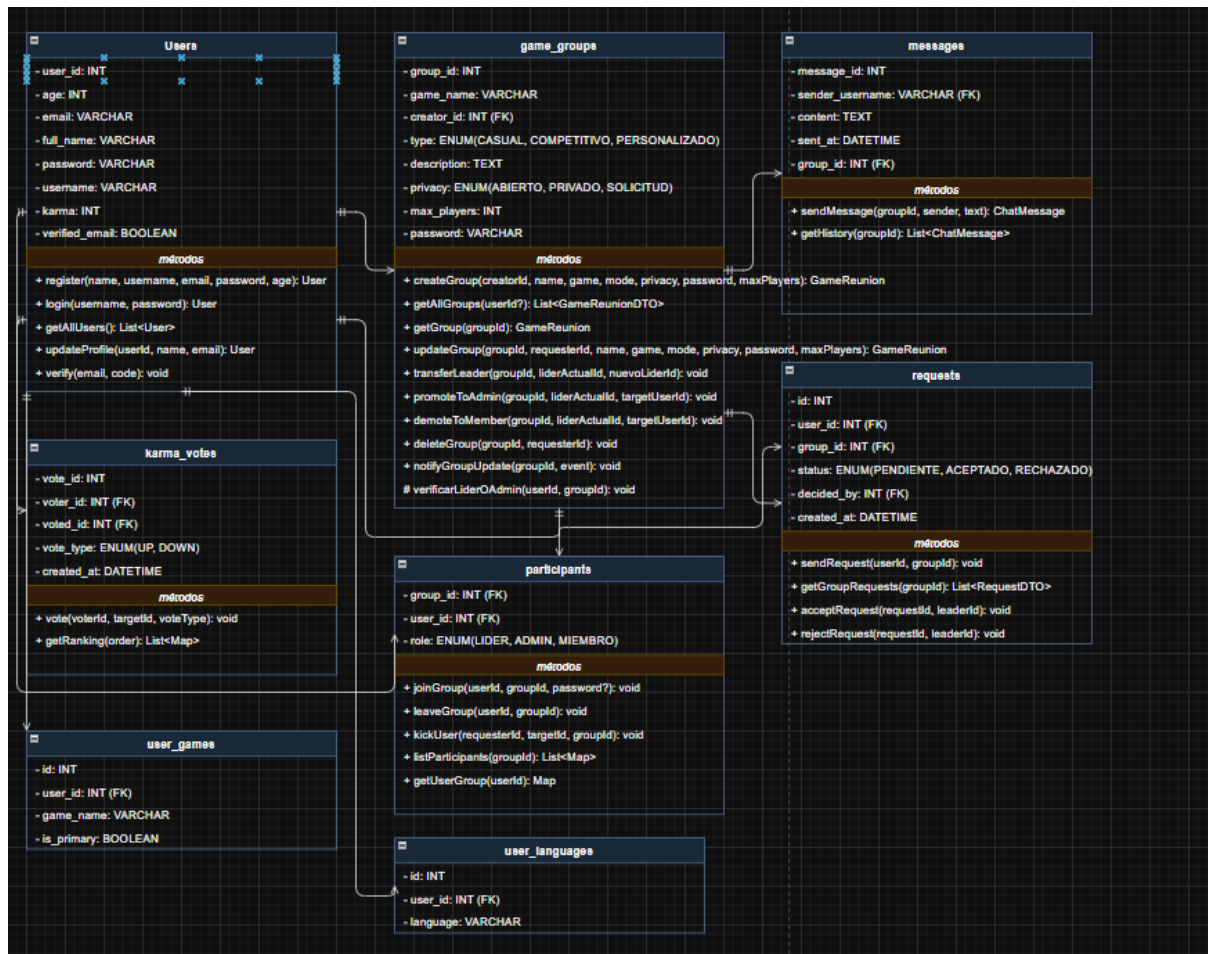
El proyecto sigue una arquitectura cliente-servidor dividida en dos partes

El **frontend** está desarrollado en Flutter y se comunica con el backend mediante dos canales: peticiones HTTP REST para operaciones como registro, login o gestión de grupos, y una conexión WebSocket persistente para la mensajería en tiempo real y las llamadas de voz grupales.

El **backend** está construido con Spring Boot y se organiza en cinco capas: el controlador expone los endpoints REST, el servicio contiene la lógica de negocio, el repositorio gestiona el acceso a los datos mediante JPA, el modelo representa las entidades de la base de datos, y la configuración centraliza los parámetros globales, CORS y WebSocket.

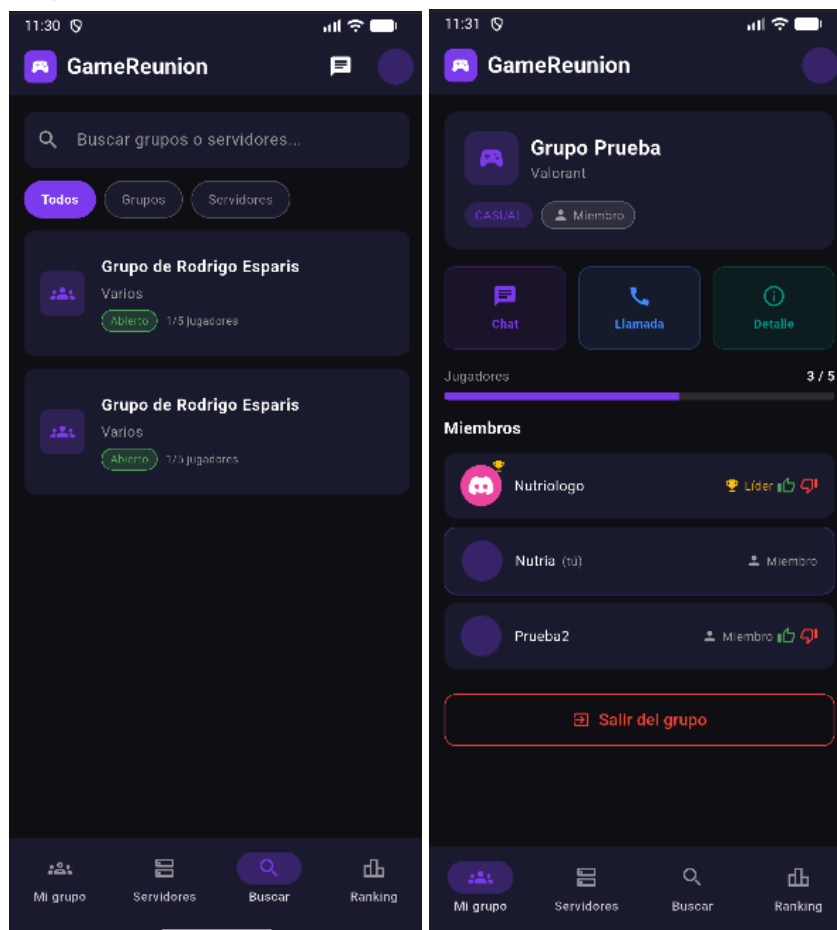
La **base de datos** es MySQL, alojada en la nube mediante Aiven, y es accedida exclusivamente desde el backend.

Diagrama de clases UML

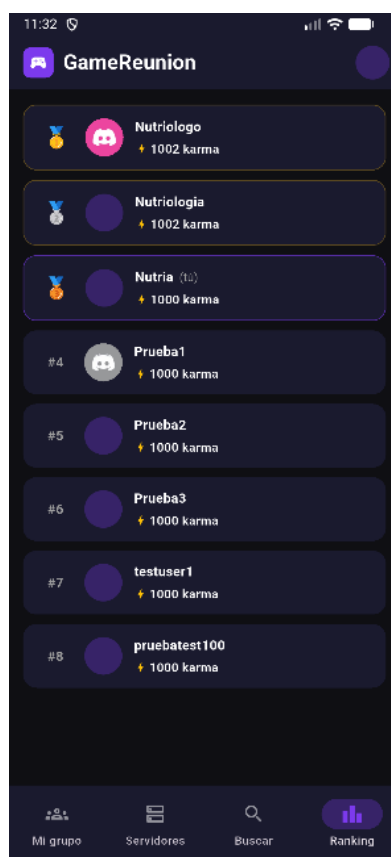


El sistema se compone de **8 entidades principales**. La clase **User** gestiona los jugadores registrados, incluyendo autenticación, perfil y sistema de karma. La clase **GameGroup** representa grupos de juego, con un líder y múltiples participantes. La relación entre User y GameGroup se hace mediante la clase **Participant**, que define el rol de cada miembro. Los mensajes se intercambian dentro de los grupos a través de la clase **Message**. Adicionalmente, el sistema gestiona solicitudes de unión (**Requests**) y votos de karma entre usuarios (**KarmaVotes**). Por último, almacenamos valores extra para la clase User en las tablas **user_Games**, **user platforms**, **user_languages**

Capturas de interfaz



En estas dos pantallas se aprecia el menú inicial para buscar grupos de otros usuarios creados, y la interfaz que se muestra al estar en un grupo con más miembros



En esta otra pantalla de interfaz se puede ver un sistema de ranking, en el que muestra los usuarios con más puntos en la parte superior, y con menos puntos en la parte inferior. Los puntos son decididos por el resto de usuarios para votar.

Decisiones de diseño

La primera decisión fue el uso de Java con Spring Boot para el backend principalmente por el dominio previo del lenguaje, lo que permitió centrarse en su lógica sin perder el tiempo en aprender nuevas tecnologías las cuales podrían no cumplir ciertos estándares, además de ser un framework ampliamente utilizado y con gran soporte.

La segunda decisión fue Flutter para frontend. Se optó por eso para que la aplicación fuera accesible desde cualquier dispositivo móvil. De esta forma cualquier usuario podría utilizarla sin que fuera exclusiva para web o escritorio, haciendo que usuarios de consolas la puedan usar.

De tercera decisión se usó WebSocket para la comunicación en tiempo real. Al mantener una conexión persistente entre clientes y servidor, resulta perfecta para chat de mensajes o para las llamadas grupales, ya que evita peticiones constantes con el servidor, y garantiza que los datos lleguen de forma inmediata a todos los miembros del grupo.

6 - Implementación

Estructura del proyecto

El proyecto está diferenciado en dos partes, el backend, y el frontend

El backend tiene su arquitectura dentro del paquete com.rodrido, organizado en cinco carpetas principales

1. Config

- a. Contiene la configuración general de la aplicación: seguridad, parámetros globales y la configuración del **WebSocket** para la comunicación en tiempo real entre usuarios.

2. Controlador

- a. Define los endpoints de la **API REST**. Agrupa los endpoints por funcionalidad (usuarios, mensajes, grupos...), recibe las peticiones HTTP y delega la lógica al servicio correspondiente.

3. Modelo

- a. Representa las entidades de la base de datos mediante clases Java anotadas con JPA/Hibernate (usuarios, mensajes, grupos, etc.).

4. Repositorio

- a. Gestiona el acceso a los datos. Extiende interfaces de Spring Data JPA para realizar operaciones CRUD y consultas personalizadas sobre la base de datos.

5. Servicio

- a. Contiene la lógica de negocio de la aplicación. Actúa como intermediario entre los controladores y los repositorios, gestionando validaciones, transformaciones y reglas de la app.

Por otro lado, el frontend está dentro de la carpeta “lib”, que es el centro de la app flutter. Dentro de esa carpeta se organiza en:

1. **Screens** (Dividida en 3 secciones)
 - a. Grupo
 - i. Pantallas relacionadas con la gestión y uso de grupos: creación, configuración, lista de miembros y chat grupal.
 - b. Iniciales
 - i. Pantallas principales de la app con la mayor parte de la funcionalidad: perfil de usuario, conversaciones y navegación central.
 - c. Registro
 - i. Flujo de autenticación: pantallas de inicio de sesión, registro de nueva cuenta y recuperación de contraseña.

(Revisar anexo 5)

Componentes principales

El primer componente son el **modelo y el repositorio**, estos representan las entidades de la base de datos y la gestión de las consultas necesarias para que los datos persistan en la base de datos

El segundo componente es la carpeta de **servicios del backend**, contiene la lógica, aquí se procesan las peticiones recibidas y coordina las operaciones sobre esos datos que se reciben

El tercer componente son los **controladores**, exponen los endpoints de la API REST, y gestionan las peticiones entrantes desde el frontend

El cuarto componente es la capa **servicios del frontend**, sirve para centrar la comunicación con el backend, tanto las peticiones HTTP, como la conexión WebSocket para mensajes y llamadas

El último componente es la **interfaz**, construida en Flutter y organizada en pantallas y widgets reutilizables, permite al usuario interactuar con la aplicación

Dificultades encontradas y soluciones

- **Configuración de WebSocket**→ Al no tener mucha experiencia con ellos, resultó un proceso difícil de hacer. Se resolvió al hacer diferentes pruebas viendo cómo afectan al programa o que requisitos piden, aparte de investigando sobre su funcionamiento
- **Gestión de archivos de configuración ocultos**→A la hora de manejar el archivo .env hubo problemas, ya que no deben subirse al repositorio las credenciales importantes, y en un principio no se había pensado esto.

Se solucionó cuando los subí a secretos de GitHub de forma segura y usando un .env.example

- **Envío de datos incorrecto entre cliente y servidor**→En varias ocasiones el frontend enviaba datos que no coinciden con lo que esperaba el backend, dando errores de respuesta. Utilizando postman y viendo que pedía concretamente cada petición se pudo ir gestionando poco a poco

7 - Multiusuario

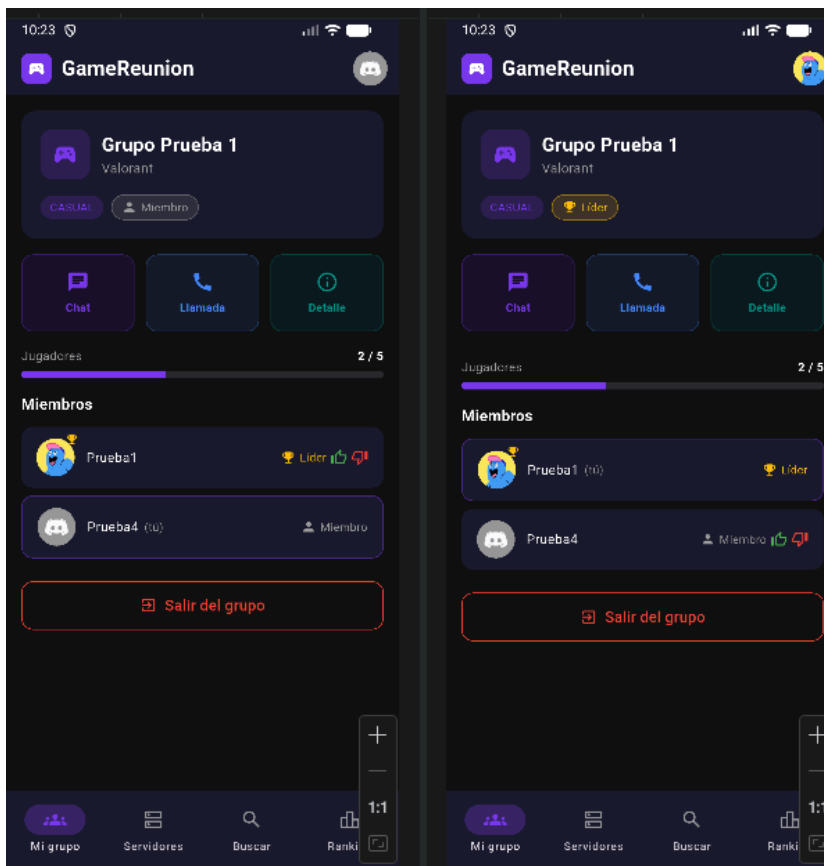
Los diferentes usuarios primero deben crear una cuenta aportando su correo electrónico, nombre, nombre de cuenta, y una contraseña.

Después podrán iniciar sesión usando su nombre de usuario y la contraseña.

Para verificar que existen dos usuarios simultáneamente deberemos tener dos dispositivos móvil android, ya sean físicos, o virtuales(Ambos tienen que ser del mismo tipo), una vez tengamos los dos dispositivos, cada uno tendrá sus credenciales de inicio de sesión, y podrán hacerlo de forma simultánea sin presentar complicación.

Por último para demostrar esta simultaneidad y conexión, al unirse a un mismo grupo, podrán ver el perfil del otro, y mandarse mensajes en tiempo real, viendo los mensajes del otro usuario en otro color(como Whatsapp, o instagram) y su nombre al lado(En caso de haber más de 2 miembros saber quién es quién)

Si la comunicación falla, pueden llegar los mensajes más tarde, enviarse con retraso, o incluso que no se envíen, haciendo que ninguno de los dos usuarios vea el mensaje.



En la imagen, se ve los dos dispositivos móviles funcionando simultáneamente, mostrando la información del Grupo Prueba 1, cada móvil está en una cuenta diferente, por lo que se muestra cuál de los usuarios es el del móvil

(Revisar anexo 4 para más pruebas)

8. Despliegue e Instalación

Manual de Instalación

Requisitos previos

Herramienta	Versión mínima	Descarga
Git	2.x	https://git-scm.com
Docker Desktop	4.x	https://www.docker.com/products/docker-desktop
Java JDK	17+	https://adoptium.net
Android Studio	Panda	https://developer.android.com/studio
Flutter SDK	3.x	https://flutter.dev/docs/get-started/install

Nota: Docker Desktop debe estar en ejecución antes de arrancar el backend.

Pasos para arrancar el proyecto

1. Clonar el repositorio

```
git clone https://github.com/rodrigoesparis/gameReunionDocker.git  
cd "repositorio deseado"
```

2. Arrancar el backend con Docker

```
docker compose up -d
```

Esto levanta el servidor Spring Boot y la base de datos MySQL. Para comprobar que todo está en marcha:

```
docker compose ps
```

Ambos servicios deben aparecer con estado **running**.

3. Arrancar la app móvil (Flutter)

Abre Android Studio, conecta un dispositivo físico o lanza un emulador Android, y desde la carpeta del proyecto Flutter:

```
cd app/ # o la carpeta donde esté el proyecto Flutter
flutter clean (Limpiar dependencias no deseadas)
flutter pub get (Importar dependencias de la app)
flutter run (
```

Asegúrate de que la URL base de la API en el código Flutter apunta a `http://10.0.2.2:8080` si usas emulador, o a la IP local del equipo si usas dispositivo físico.

Configuración

El proyecto utiliza un fichero `.env` para almacenar todas las variables sensibles (credenciales, cadenas de conexión, etc.). Este fichero **no se incluye en el repositorio** por seguridad.

Para crearlo, se proporciona un fichero `.env.example` como plantilla. Basta con copiarlo y rellenar los valores propios:

El fichero `.env.example` tiene este aspecto:

```
env
# Base de datos
MYSQL_DATABASE=nombre_bbdd
MYSQL_USER=usuario
MYSQL_PASSWORD=contraseña
MYSQL_ROOT_PASSWORD=contraseña_root

# Spring Boot
SPRING_DATASOURCE_URL=jdbc:mysql://db:3306/nombre_bbdd
SPRING_DATASOURCE_USERNAME=usuario
SPRING_DATASOURCE_PASSWORD=contraseña
```

Docker Compose carga automáticamente el `.env` al ejecutar `docker compose up`, por lo que no es necesario ningún paso adicional. Las variables definidas en él son inyectadas tanto en el contenedor de MySQL como en el de Spring Boot.

Datos de ejemplo

Al descargar el repositorio, se encuentra una carpeta con los datos de prueba llamada "Scripts".

En ella se encuentran diferentes tablas con sus respectivos datos de prueba por defecto. Existen varias formas de ejecutarlos:

1. Se pueden ejecutar individualmente en la base de datos para crear los datos de prueba y utilizarlos
2. Usando Docker Desktop, los datos de prueba se importan automáticamente en la base de datos sin ningún paso intermedio

Manual de Usuario

Primer acceso

1. Arrancar el backend (ver sección 8.1).
2. Abrir la app en el móvil o emulador.
3. En la pantalla de inicio, registrar una cuenta nueva o iniciar sesión con las credenciales de prueba incluidas en el seed:
 - Usuario: prueba1@gmail.com
 - Contraseña: Test1234!
4. Revisar la cuenta de correo electrónico para confirmar la cuenta mediante un código de verificación

Funcionalidades principales

1. En la pantalla principal una vez iniciada sesión, podrás unirte a diferentes grupos de otros usuarios.
2. En la pantalla "Mi grupo", podrás ver el grupo en el que estás en ese momento, mostrando los usuarios que están contigo, y una interfaz de mensajes.

En caso de que no estés en ningún grupo tendrás dos botones

- a. "Crear grupo", con este botón te mandará a una pestaña para crear tu propio grupo, y esperar a que se una gente
 - b. "Explorar grupo", te mandará a la pestaña principal, donde verás los grupos disponibles
3. En la pantalla "Ranking", se mostrará un listado de todos los usuarios de la app y sus puntos de "Karma", ordenados de más puntos a menos, pudiendo cambiar este orden en el botón superior
 4. Pinchando en el círculo superior derecho (Perfil), podrás ver tu información del perfil, pudiendo modificar valores informativos, tu foto de perfil, o cerrar sesión de la cuenta.

(Revisar anexo 3 para fotos y pruebas)

9 - Pruebas

Plan de pruebas

Para las pruebas se utilizó un dispositivo móvil virtual, verificando que la aplicación funcionara debidamente y sacará los datos correctos por consola, aparte de que sea visualmente intuitiva, y que los botones, accionadores, o textos se mostrarán de forma correcta y con un tamaño adecuado.

Por otro lado también se utilizó PostMan, para verificar que los datos del backend que se mandaban fueran correctos y completos, aparte de estar bien estructurados y no hubiera duplicados

Casos de prueba

ID	Funcionalidad	Acción realizada	Resultado esperado	Resultado obtenido	✓ / ✗
C P-01	Registro	Registrar usuario con datos válidos	Cuenta creada y acceso a la app	Cuenta creada correctamente	✓
C P-02	Registro	Registrar con correo ya existente	Mensaje de error	Error mostrado al usuario	✓
C P-03	Login	Iniciar sesión con credenciales correctas	Acceso a pantalla principal	Acceso concedido	✓
C P-04	Login	Iniciar sesión con contraseña incorrecta	Mensaje de error	Error mostrado al usuario	✓
C P-05	Grupos	Crear un grupo con datos válidos	Grupo creado, usuario asignado como líder	Grupo creado correctamente	✓
C P-06	Grupos	Solicitar unirse a un grupo existente	Solicitud enviada al administrador	Solicitud registrada	✓
C P-07	Mensajería	Enviar un mensaje en el chat del grupo	Mensaje visible para todos en tiempo real	Mensaje recibido en tiempo real	✓
C P-08	Mensajería	Enviar mensaje vacío	El sistema impide el envío	Envío bloqueado	✓
C P-09	Llamada de voz	Unirse a la llamada grupal	Usuario conectado a la sesión de voz	Conexión establecida correctamente	✓
C P-10	Multiusuario	Dos usuarios en el mismo grupo enviando mensajes	Ambos ven los mensajes en tiempo real	Mensajes recibidos correctamente	✓

10 - Resultados, conclusiones y vías futuras

Objetivos alcanzados

Conseguí cumplir la mayoría de los objetivos. Se implementaron las funciones principales de la app, junto con funciones extra que fui pensando durante el desarrollo, incluyendo registros con seguridad, conexiones reales, y datos ocultos para usuarios externos.

Quedó incompleto un apartado de servidores, el cual se plantea para añadir en una mejora futura para completar. Esto junto con futuras ideas, la aplicación puede seguir creciendo para añadir funcionalidades que ayuden a mejorar la aplicación para que los usuarios estén contentos con el resultado.

A nivel de organización y proceso de trabajo, se cumplieron los objetivos planteados. Empezando con un desarrollo con antelación, evitando dejar trabajo para días al final, lo que me permitió resolver los problemas con calma y sin prisa. Mantuve una rutina constante, de trabajar todas las semanas, ya sea en la aplicación, en la documentación, o en ideas.

Por otro lado se llevó un registro de ideas, progreso, y tareas complicadas y por cumplir, lo que ayudó a entender que me faltaba o que podía gestionar mejor.

Conclusiones

Este proyecto me ayudó a mejorar en conocimientos a los que ya estaba acostumbrado como a usar Flutter o Spring Boot, y conocer otros como las bases de datos en la nube, los ficheros .env, o la gestión de archivos de aplicaciones para su lanzamiento.

La constancia de trabajar todos los días con antelación para presentar un proyecto más organizado y mejor realizado es importante de mencionar también.

También mejoré en ámbito de ser autodidacta y aprender mediante tus propios medios para realizar tus proyectos o trabajos.

Vías futuras/Mejoras

- Completar la sección de servidores que quedó pendiente durante el desarrollo
- Funciones específicas para cada juego, como sistema de voz por turnos o apagar el micrófono automáticamente para juegos como Among Us, o mostrar que usuario esta hablando para identificar voces
- Mejorar el sistema de karma, permitiendo que los usuarios puedan votar más veces a cada usuario con el paso del tiempo, sistema de bonificación(10 likes sin ningun dislike suma extra)
- Versión para iOS y aplicación de escritorio para ordenador
- Notificaciones push para ciertas acciones

11 - Glosario

API REST: Interfaz de comunicación entre frontend y backend para enviar y recibir datos

APK: Formato de archivo de instalación para aplicaciones en dispositivos Android

Backend: Parte del sistema que funciona en el servidor, encargada de la lógica, la base de datos y gestión

Dart: Lenguaje de programación desarrollado por Google

Docker : Herramienta para empaquetar una aplicación con todas sus dependencias en un contenedor, haciendo que su despliegue sea más sencillo

.env: Archivo de configuración con variables sensibles

Flutter: Framework de Google, que permite desarrollar aplicaciones móviles para android e iOS

Frontend: Parte de la aplicación que ve e interactúa el usuario

Karma: Sistema de puntuación dentro de la app, que refleja las valoraciones

Spring Boot: Framework de java que facilita la creación de API 's REST y aplicaciones backend de forma rápida y estructurada.

WebSocket: Protocolo de comunicación que mantiene una conexión persistente entre cliente y servidor

12 - Bibliografía/Webgrafia

- Documentación de Aiven - Para información útil en conexiones para la base de datos con el workspace y en backend
(<https://aiven.io/docs/>)
- Documentación de Flutter - Utilización de contenedores, botones, widgets, formatos, o instalación
(<https://docs.flutter.dev/>)
- Discord - Ayuda visual para diseñar la interfaz y la temática
(<https://discord.com/>)
- Figma - Crear el diseño visual
(<https://www.figma.com/es-es/>)
- GitHub - Control de versiones
(<https://github.com/>)
- Mailtrap - Verificación de correos de prueba para multiples usuarios
(<https://mailtrap.io/es/>)
- Dbdiagram - Creación de diagramas
(<https://dbdiagram.io/>)
- Claude AI - Herramienta multiusos
(<https://platform.claude.com/login?returnTo=%2F%3F>)

ANEXOS

Anexo 1 - Casos de Uso

1. Usuario No Registrado

CU-01 – Registrarse en la aplicación	
Actor(es)	Usuario no registrado
Descripción	El usuario crea una nueva cuenta proporcionando su correo electrónico, nombre completo, nombre de usuario y contraseña.
Precondición	El usuario no tiene una cuenta registrada en el sistema.
Postcondición	Se crea una nueva cuenta de usuario en el sistema. El usuario pasa a estar registrado.
Flujo principal	1. El usuario accede a la pantalla de registro. 2. Introduce correo, nombre, nombre de usuario y contraseña. 3. Confirma los datos y envía el formulario. 4. El sistema valida los datos y crea la cuenta. 5. El usuario recibe confirmación de registro exitoso.
Flujo alternativo	3a. El usuario cancela el registro: el sistema vuelve a la pantalla de inicio sin crear la cuenta.
Excepciones	E1: El correo ya está registrado → el sistema muestra un mensaje de error. E2: El nombre de usuario ya está en uso → el sistema solicita otro nombre de usuario. E3: La contraseña no cumple los requisitos mínimos → el sistema informa al usuario.

CU-02 – Iniciar sesión	
Actor(es)	Usuario registrado
Descripción	El usuario accede al sistema introduciendo su nombre de usuario y contraseña.
Precondición	El usuario tiene una cuenta registrada en el sistema.
Postcondición	El usuario queda autenticado y accede a la pantalla principal de la aplicación.

Flujo principal	1. El usuario accede a la pantalla de inicio de sesión. 2. Introduce su nombre de usuario y contraseña. 3. El sistema valida las credenciales. 4. El usuario accede a la aplicación.
Flujo alternativo	2a. El usuario cancela el inicio de sesión: el sistema vuelve a la pantalla de inicio.
Excepciones	E1: Credenciales incorrectas → el sistema muestra un mensaje de error. E2: La cuenta no existe → el sistema informa al usuario.

2. Miembro

CU-03 – Unirse a un grupo existente	
Actor(es)	Miembro
Descripción	El usuario envía una solicitud para unirse a un grupo existente en la aplicación.
Precondición	El usuario está autenticado y no pertenece al grupo al que desea unirse.
Postcondición	Se registra la solicitud de entrada del usuario en el sistema. El grupo recibe la notificación.
Flujo principal	1. El usuario busca o selecciona un grupo. 2. Accede al detalle del grupo y pulsa 'Solicitar unirse'. 3. El sistema registra la solicitud y la notifica al administrador o líder del grupo. 4. El sistema confirma al usuario que su solicitud ha sido enviada.

Flujo alternativo	2a. El grupo está lleno → el sistema informa al usuario de que no hay plazas disponibles.
Excepciones	E1: El usuario ya ha enviado una solicitud pendiente → el sistema informa que ya existe una solicitud activa.

CU-04 – Abandonar un grupo	
Actor(es)	Miembro
Descripción	El usuario abandona voluntariamente el grupo en el que se encuentra.
Precondición	El usuario está autenticado y pertenece al grupo.
Postcondición	El usuario deja de ser miembro del grupo. El grupo actualiza su lista de miembros.
Flujo principal	1. El usuario accede al grupo del que quiere salir. 2. Selecciona la opción 'Abandonar grupo'. 3. El sistema solicita confirmación. 4. El usuario confirma. 5. El sistema elimina al usuario del grupo y actualiza la lista de miembros.
Flujo alternativo	4a. El usuario cancela la acción: no se produce ningún cambio.
Excepciones	E1: El usuario es el líder del grupo → el sistema impide la acción e indica que debe transferir el liderazgo antes de abandonar.

CU-05 – Enviar y recibir mensajes en tiempo real	
Actor(es)	Miembro
Descripción	El usuario envía mensajes de texto y recibe los mensajes de otros miembros dentro del chat del grupo en tiempo real.
Precondición	El usuario está autenticado y es miembro del grupo.
Postcondición	El mensaje queda registrado y es visible para todos los miembros del grupo en tiempo real.
Flujo principal	1. El usuario accede al chat del grupo. 2. Escribe un mensaje de texto en el campo de entrada. 3. Pulsa 'Enviar'. 4. El sistema distribuye el mensaje en tiempo real a todos los miembros conectados. 5. Los miembros reciben y visualizan el mensaje.
Flujo alternativo	2a. El usuario decide no enviar el mensaje y borra el contenido del campo.
Excepciones	E1: El usuario pierde la conexión → el sistema intenta reenviar el mensaje cuando se recupere la conexión. E2: El mensaje está vacío → el sistema impide el envío.

CU-06 – Unirse y abandonar llamada de voz	
Actor(es)	Miembro
Descripción	El usuario se une a la llamada de voz activa del grupo y puede abandonarla cuando lo desee.
Precondición	El usuario está autenticado y es miembro del grupo.
Postcondición	El usuario participa en la llamada de voz del grupo / El usuario abandona la llamada.
Flujo principal	1. El usuario accede al grupo y visualiza la llamada de voz activa. 2. Pulsa 'Unirse a la llamada'. 3. El sistema conecta al usuario a la sesión de voz del grupo. 4. El usuario participa en la llamada. 5. Cuando desea salir, pulsa 'Abandonar llamada'. 6. El sistema desconecta al usuario de la sesión de voz.
Flujo alternativo	2a. No hay llamada activa → el sistema muestra que no hay ninguna sesión en curso.
Excepciones	E1: El dispositivo del usuario no tiene micrófono disponible → el sistema informa del error de hardware. E2: Fallo de conexión → el usuario es desconectado automáticamente de la llamada.

CU-07 – Crear un grupo	
Actor(es)	Miembro
Descripción	El usuario crea un nuevo grupo configurando su nombre, el juego asociado y el número máximo de jugadores.
Precondición	El usuario está autenticado en el sistema.
Postcondición	Se crea un nuevo grupo. El usuario creador pasa a ser el líder del grupo.
Flujo principal	1. El usuario accede a la opción 'Crear grupo'. 2. Introduce el nombre del grupo, selecciona el juego y establece el número máximo de jugadores. 3. Confirma la creación del grupo. 4. El sistema registra el grupo y asigna al usuario como líder. 5. El sistema redirige al usuario a la pantalla del grupo recién creado.
Flujo alternativo	3a. El usuario cancela: el sistema vuelve a la pantalla anterior sin crear el grupo.
Excepciones	E1: El nombre del grupo ya existe → el sistema solicita un nombre diferente. E2: Datos obligatorios incompletos → el sistema señala los campos que faltan.

3. Administrador

CU-08 – Aceptar o rechazar solicitudes de entrada	
Actor(es)	Administrador, Líder
Descripción	El administrador gestiona las solicitudes de entrada al grupo, aceptando o rechazando a los usuarios solicitantes.
Precondición	El usuario está autenticado, tiene rol de administrador o líder en el grupo, y existen solicitudes pendientes.
Postcondición	La solicitud queda resuelta. El solicitante es admitido en el grupo o su solicitud es rechazada.
Flujo principal	1. El administrador accede a la lista de solicitudes pendientes del grupo. 2. Selecciona una solicitud. 3. Elige 'Aceptar' o 'Rechazar'. 4. El sistema actualiza el estado de la solicitud y notifica al solicitante del resultado.
Flujo alternativo	3a. El administrador no toma ninguna acción: la solicitud permanece pendiente.
Excepciones	E1: El grupo ya ha alcanzado el número máximo de miembros → el sistema impide aceptar nuevas solicitudes.

CU-09 – Expulsar miembros del grupo	
Actor(es)	Administrador, Líder
Descripción	El administrador expulsa a un miembro del grupo. No puede expulsar a otros administradores ni al líder.
Precondición	El usuario está autenticado con rol de administrador. El miembro objetivo tiene rol de miembro (no admin ni líder).
Postcondición	El miembro expulsado es eliminado del grupo y notificado.
Flujo principal	1. El administrador accede a la lista de miembros del grupo. 2. Selecciona al miembro que desea expulsar. 3. Elige la opción 'Expulsar'. 4. El sistema solicita confirmación. 5. El administrador confirma. 6. El sistema elimina al miembro del grupo y le notifica.
Flujo alternativo	5a. El administrador cancela la acción: no se produce ningún cambio.
Excepciones	E1: El miembro objetivo es administrador o líder → el sistema deniega la acción e informa del motivo.

CU-10 – Ascender el rol de un miembro	
Actor(es)	Administrador, Líder

Descripción	El administrador asciende el rol de un miembro del grupo a administrador.
Precondición	El usuario tiene rol de administrador o líder en el grupo. El miembro objetivo tiene rol de miembro.
Postcondición	El miembro seleccionado pasa a tener rol de administrador dentro del grupo.
Flujo principal	1. El administrador accede a la lista de miembros. 2. Selecciona al miembro al que desea ascender. 3. Elige 'Ascender a administrador'. 4. El sistema solicita confirmación. 5. El administrador confirma. 6. El sistema actualiza el rol del miembro y lo notifica.
Flujo alternativo	5a. El administrador cancela: no se produce ningún cambio.
Excepciones	E1: El miembro ya tiene el rol de administrador → el sistema informa que el rol ya está asignado.

4. Líder

CU-11 – Gestionar el grupo sin restricciones	
Actor(es)	Líder
Descripción	El líder puede realizar cualquier acción de gestión sobre el grupo, incluyendo las acciones de administrador y otras adicionales sin restricciones de rol.
Precondición	El usuario está autenticado y tiene rol de líder en el grupo.
Postcondición	Las acciones de gestión solicitadas se aplican correctamente al grupo.
Flujo principal	1. El líder accede a la configuración o panel de gestión del grupo. 2. Realiza las acciones de gestión necesarias (editar configuración, gestionar miembros, etc.). 3. El sistema aplica los cambios y confirma al líder.
Flujo alternativo	2a. El líder decide no realizar cambios y cierra el panel de gestión.
Excepciones	E1: Fallo del sistema al guardar los cambios → el sistema informa del error y solicita reintentar.

CU-12 – Cerrar o eliminar un grupo	
Actor(es)	Líder
Descripción	El líder cierra temporalmente o elimina definitivamente el grupo.
Precondición	El usuario está autenticado y tiene rol de líder en el grupo.

Postcondición	El grupo queda cerrado o eliminado. Todos los miembros son notificados.
Flujo principal	1. El líder accede a la configuración del grupo. 2. Selecciona 'Cerrar grupo' o 'Eliminar grupo'. 3. El sistema solicita confirmación. 4. El líder confirma la acción. 5. El sistema cierra o elimina el grupo y notifica a todos los miembros.
Flujo alternativo	4a. El líder cancela: no se produce ningún cambio.
Excepciones	E1: Error interno al eliminar el grupo → el sistema informa del error y no se completa la eliminación.

CU-13 – Transferir el liderazgo	
Actor(es)	Líder
Descripción	El líder transfiere su rol de liderazgo a otro miembro del grupo.
Precondición	El usuario está autenticado con rol de líder. Existe al menos otro miembro en el grupo.
Postcondición	El miembro seleccionado pasa a ser el nuevo líder. El líder anterior pasa a tener rol de miembro o administrador.
Flujo principal	1. El líder accede a la lista de miembros del grupo. 2. Selecciona al miembro al que desea transferir el liderazgo. 3. Elige 'Transferir liderazgo'. 4. El sistema solicita confirmación. 5. El líder confirma. 6. El sistema actualiza los roles y notifica a los afectados.
Flujo alternativo	5a. El líder cancela: no se produce ningún cambio.
Excepciones	E1: El miembro seleccionado ya no forma parte del grupo → el sistema informa del error y solicita seleccionar otro miembro.

Anexo 2 - Casos de prueba

CP-01

```

PS C:\Users\Usuario> Invoke-WebRequest -Uri "http://localhost:8080/users/register" -Method POST -Headers @{"Content-Type"="application/json"} -Body '{"name":"Prueba Test","username":"prubatest100","email":"prubatest100@gmail.com","password":"Test1234!","age":20}' -UseBasicParsing

StatusCode      : 200
StatusDescription : 
Content         : {"idUser":8,"name":"Prueba Test","username":"prubatest100","email":"prubatest100@gmail.com","password":"Test1234!","age":20,"karma":1000,"country":null,"timezone":null,"bio":null,"callStyle":null,"p...
RawContent      : HTTP/1.1 200
                  Vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
                  Transfer-Encoding: chunked
                  Content-Type: application/json
                  Date: Fri, 05 Jun 2026 09:45:39 GMT

                  {"idUser":8...
Forms           : 
Headers         : {[Vary, Origin,Access-Control-Request-Method,Access-Control-Request-Headers], [Transfer-Encoding, chunked], [Content-Type, application/json], [Date, Fri, 05 Jun 2026 09:45:39 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : 
RawContentLength : 256

```

CP-02

```

PS C:\Users\Usuario> Invoke-WebRequest -Uri "http://localhost:8080/users/register" -Method POST -Headers @{"Content-Type"="application/json"} -Body '{"name":"Prueba Test","username":"prubatest100","email":"prubatest100@gmail.com","password":"Test1234!","age":20}' -UseBasicParsing
Invoke-WebRequest : El username 'prubatest100' ya está en uso.
En línea: 1 Carácter: 1
+ Invoke-WebRequest -Uri "http://localhost:8080/users/register" -Method ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand

```

CP-03

```

PS C:\Users\Usuario> Invoke-WebRequest -Uri "http://localhost:8080/users/login" -Method POST -Headers @{"Content-Type"="application/json"} -Body '{"username":"Nutriologo","password":"Test1234!"}' -UseBasicParsing

StatusCode      : 200
StatusDescription : 
Content         : {"idUser":1,"name":"Rodrigo Esparis","username":"Nutriologo","email":"rodrigoesparis@gmail.com","password":"Test1234!","age":20,"karma":1002,"country":"Espana","timezone":"Madrid","bio":"Soy gaimer pr...
RawContent      : HTTP/1.1 200
                  Vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
                  Transfer-Encoding: chunked
                  Content-Type: application/json
                  Date: Fri, 05 Jun 2026 09:48:47 GMT

                  {"idUser":1...
Forms           : 
Headers         : {[Vary, Origin,Access-Control-Request-Method,Access-Control-Request-Headers], [Transfer-Encoding, chunked], [Content-Type, application/json], [Date, Fri, 05 Jun 2026 09:48:47 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : 
RawContentLength : 6758

```

CP-04

```

PS C:\Users\Usuario> Invoke-WebRequest -Uri "http://localhost:8080/users/login" -Method POST -Headers @{"Content-Type"="application/json"} -Body '{"username":"Nutriologo","password":"incorrecta123"}' -UseBasicParsing
Invoke-WebRequest : Contraseña incorrecta.
En línea: 1 Carácter: 1
+ Invoke-WebRequest -Uri "http://localhost:8080/users/login" -Method PO ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand

```

CP-05

```

PS C:\Users\Usuario> Invoke-WebRequest -Uri "http://localhost:8080/groups" -Method POST -Headers @{"Content-Type"="application/json"} -Body '{"creatorId":1,"name":"Grupo Prueba","game":"Valorant","mode":"CASUAL","privacy":"ABIERTO","password":null,"maxPlayers":5}' -UseBasicParsing

StatusCode      : 200
StatusDescription :
Content         : {"idGroup":17,"name":"Grupo Prueba","game":"Valorant","mode":"CASUAL","privacy":"ABIERTO","password":null,"maxPlayers":5,"creator":{"idUser":1,"name":"Rodrigo Esparis","username":"Nutriologo","email":...
RawContent      : HTTP/1.1 200
                  Vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
                  Transfer-Encoding: chunked
                  Content-Type: application/json
                  Date: Fri, 05 Jun 2026 09:53:56 GMT

                  {"idGroup":...
Forms           :
Headers         : {[Vary, Origin,Access-Control-Request-Method,Access-Control-Request-Headers], [Transfer-Encoding, chunked], [Content-Type, application/json], [Date, Fri, 05 Jun 2026 09:53:56 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      :
RawContentLength : 6903

```

CP-06

```

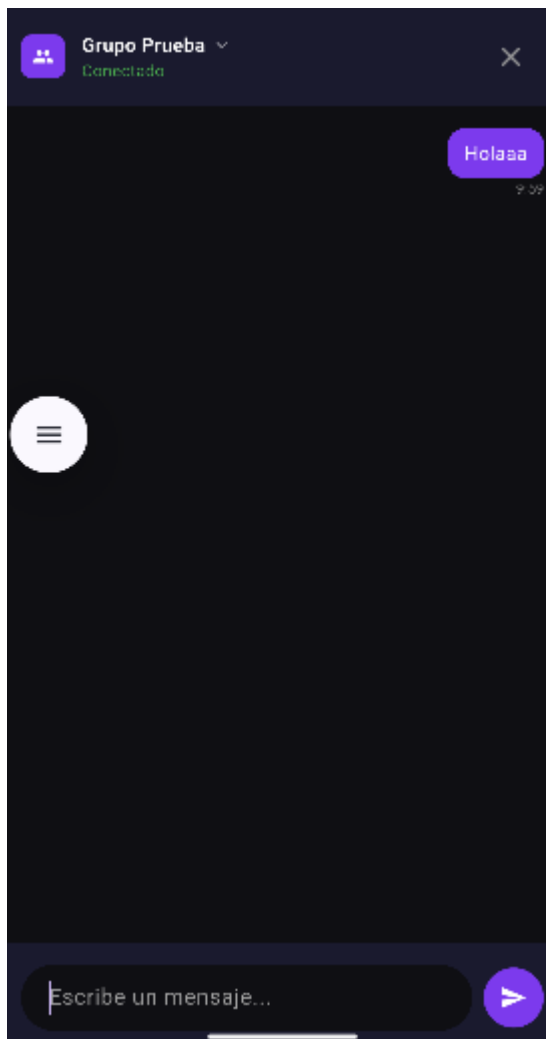
PS C:\Users\Usuario> Invoke-WebRequest -Uri "http://localhost:8080/participants/join?userId=5&groupId=17" -Method POST -UseBasicParsing

StatusCode      : 200
StatusDescription :
Content         : Te has unido al grupo correctamente.
RawContent      : HTTP/1.1 200
                  Vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
                  Content-Length: 36
                  Content-Type: text/plain; charset=UTF-8
                  Date: Fri, 05 Jun 2026 09:54:52 GMT

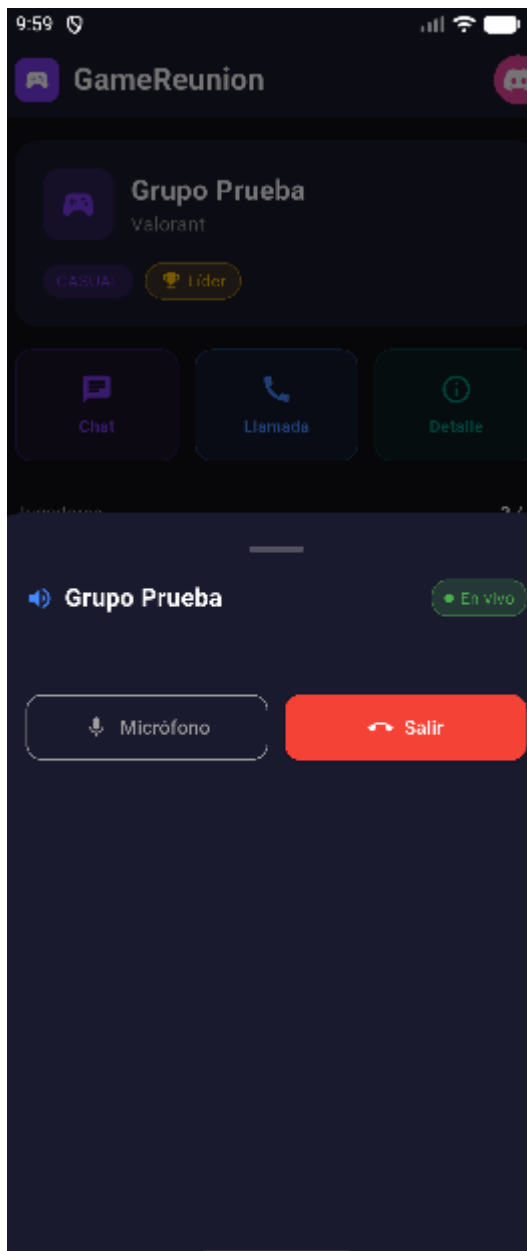
                  Te has unid...
Forms           :
Headers         : {[Vary, Origin,Access-Control-Request-Method,Access-Control-Request-Headers], [Content-Length, 36], [Content-Type, text/plain; charset=UTF-8], [Date, Fri, 05 Jun 2026 09:54:52 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      :
RawContentLength : 36

```

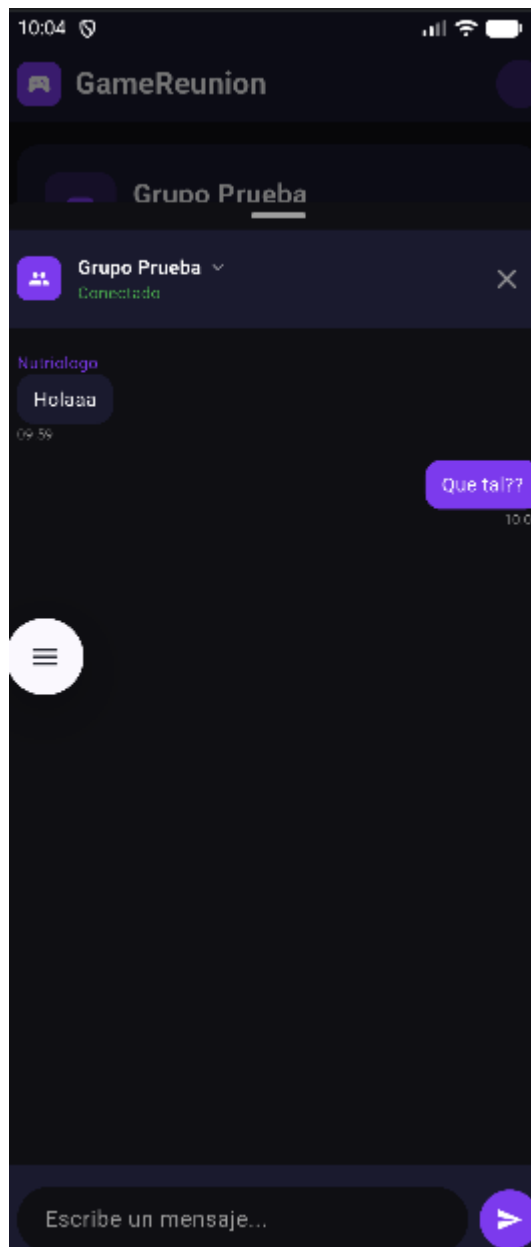
CP-07



CP-09



CP-10



Anexo 3 - Manual de usuario

Pantalla principal

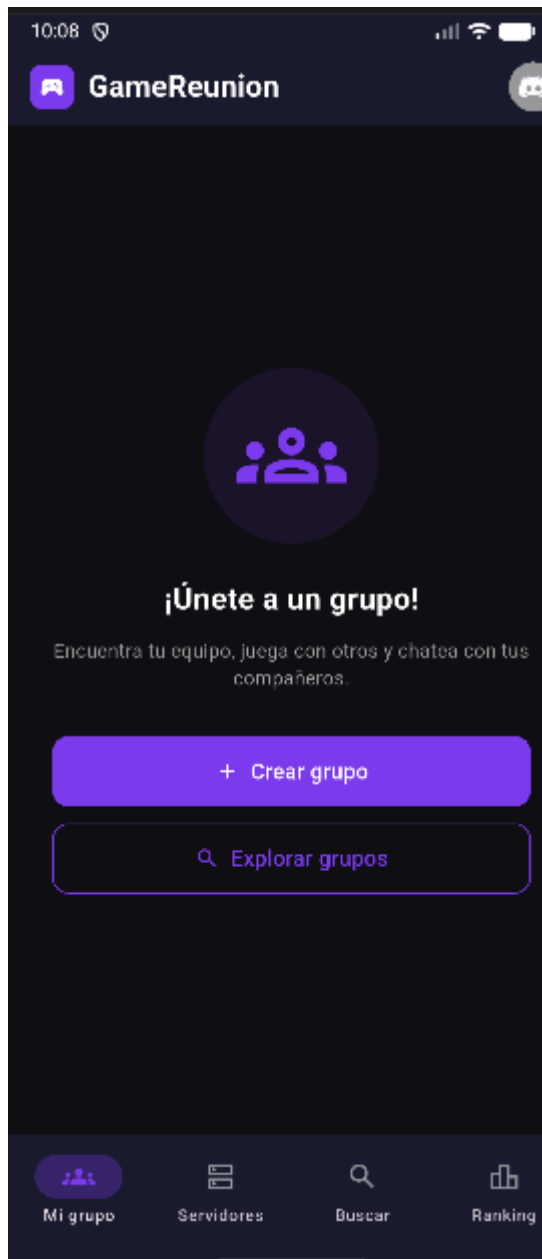


En la parte inferior está la selección de distintas pantallas

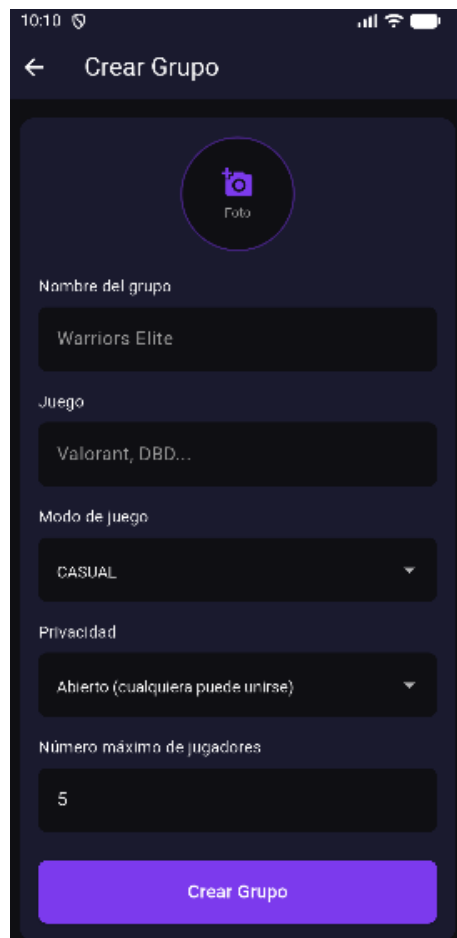
Se muestra un listado con los grupos disponibles de otros usuarios, su cantidad de jugadores, su privacidad, y el juego al que están jugando.

Tiene también un menú de búsqueda, y filtros.

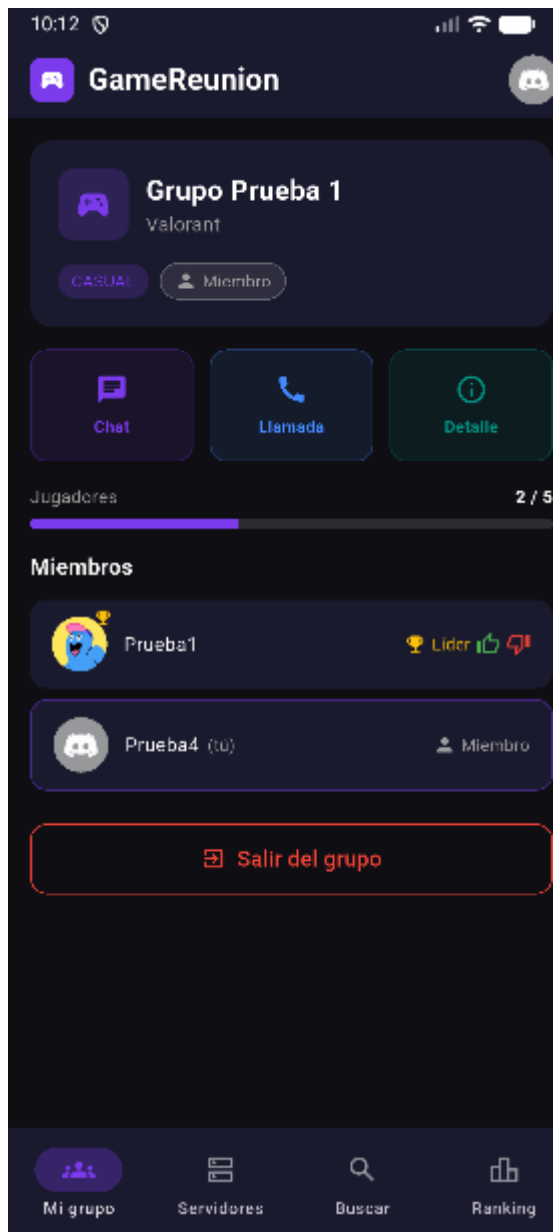
Mi grupo (Grupo no existente)



En esta ventana, al no estar en ningún grupo, mostrará los dos botones disponibles, uno para la creación del nuevo grupo(imagen inferior), donde se podrá cambiar los valores que se deseen para cada situación
El segundo botón de explorar, regresará a la pantalla principal

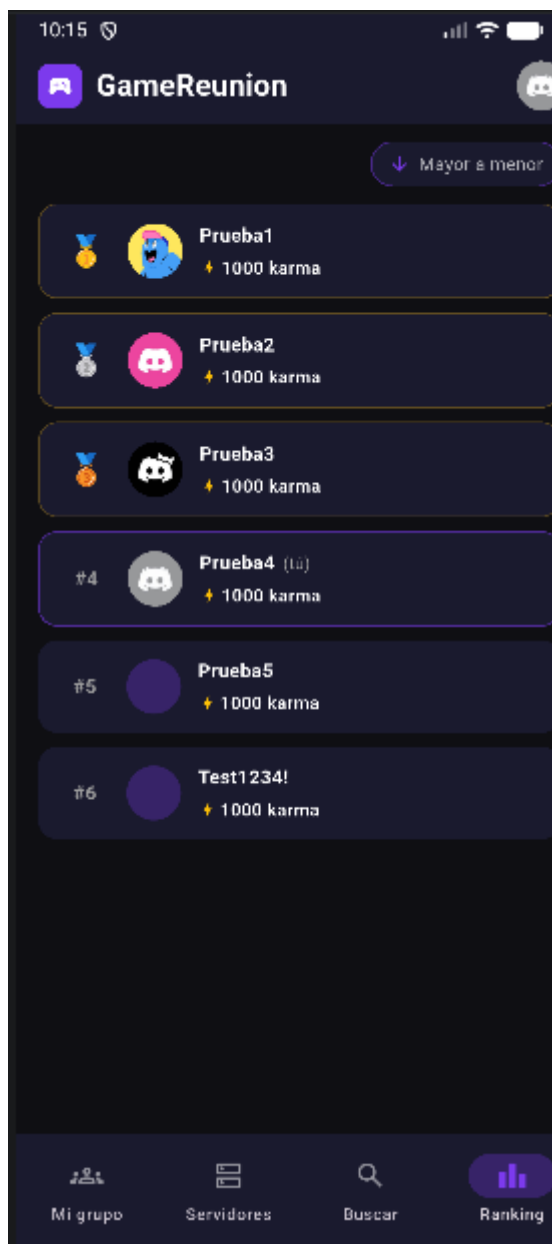


Mi grupo (Grupo existente)



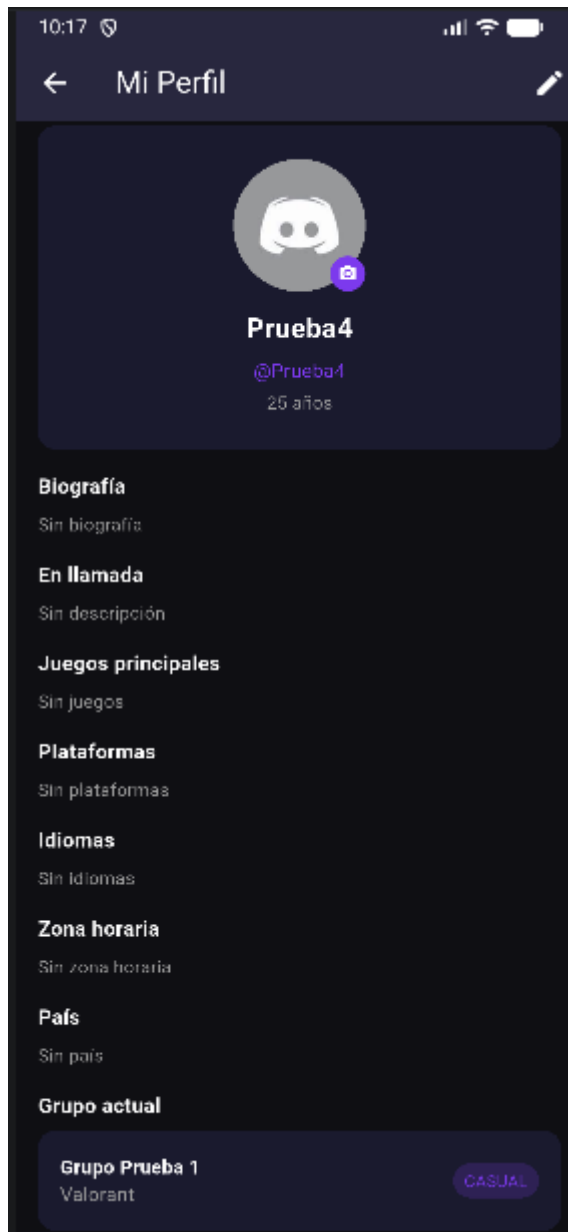
En caso de formar parte de un grupo, se mostrará esta interfaz, donde se podrá ver información básica del grupo, junto con los usuarios, y un apartado de mensajes y llamada

Ranking



En la última ventana, se mostrará un ranking de usuarios según su puntuación de karma, esta puntuación se puede mostrar de mayor a menor, o de menor a mayor según guste.

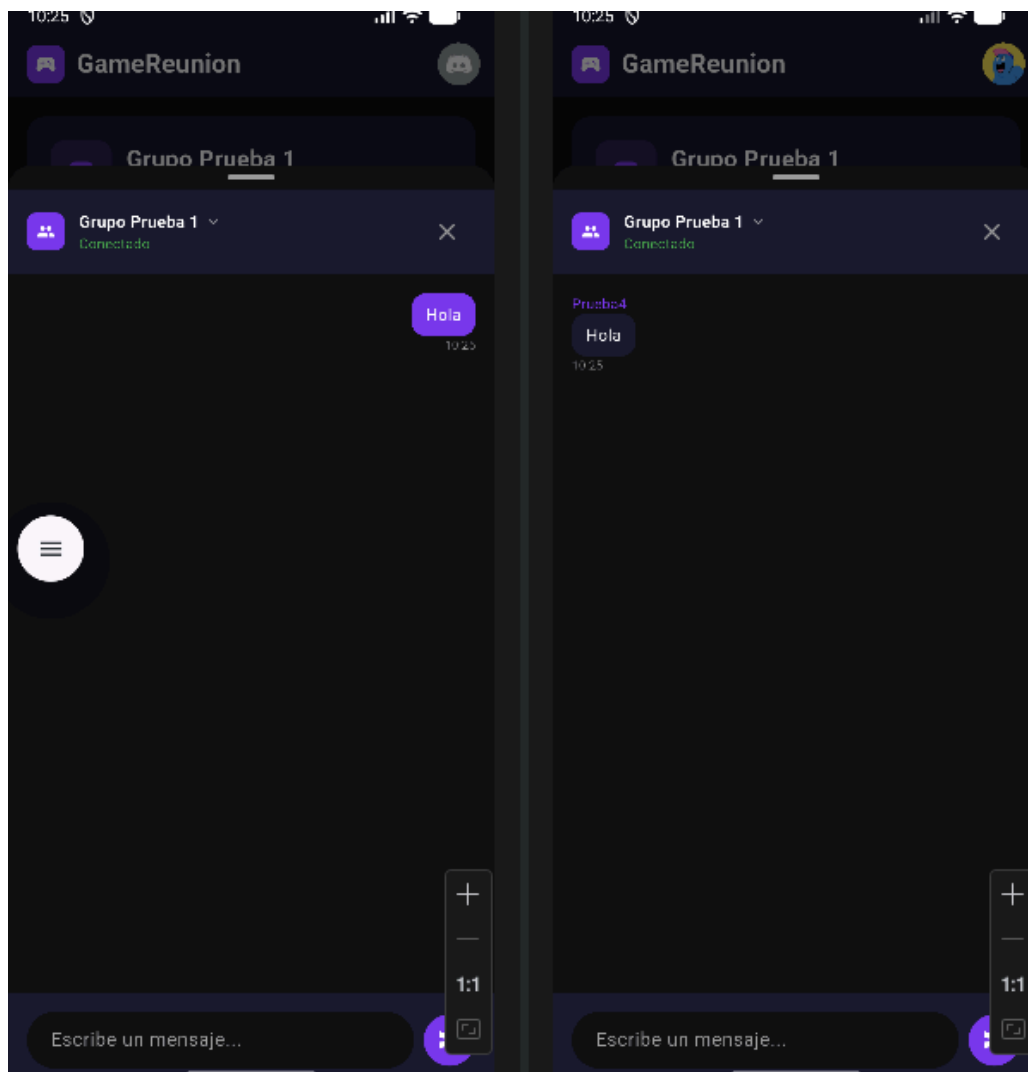
Perfil



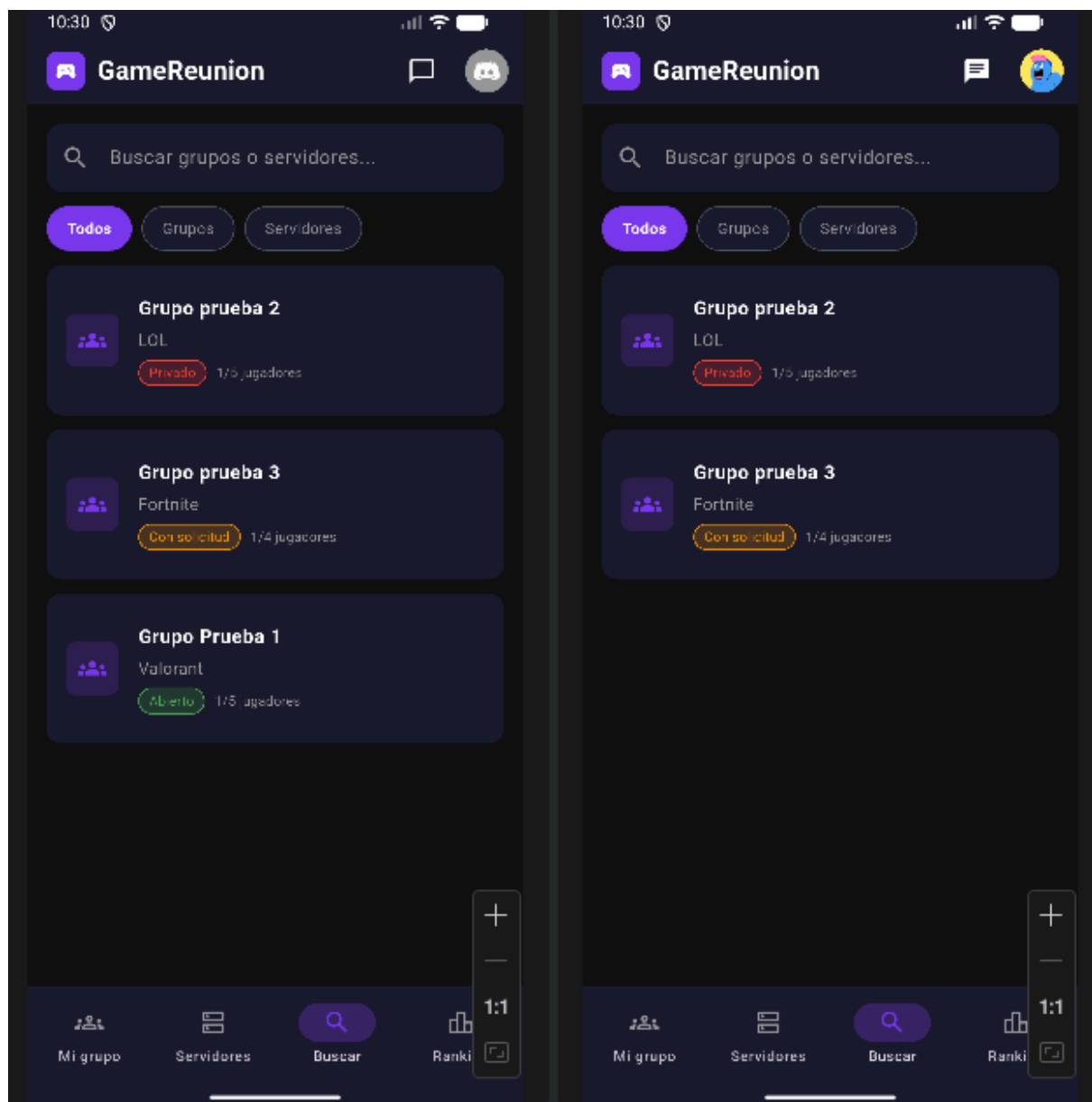
En el botón de perfil, podremos ver información básica de nuestro usuario, como el nombre, la edad, o valores que podremos cambiar, como la biografía, los juegos, o las plataformas.

Para cambiar estos valores, dándole al lápiz en la parte superior derecha, los campos entrarán en modo edición, y podrás cambiar cada uno de ellos a gusto.

Anexo 4 - Multiusuario concurrente



En esta segunda imagen, se ve como los mensajes de cada usuario se muestran de forma distinta, para representar los mensajes propios, o externos, mostrando también el nombre de usuario, para saber de quién recibes mensajes.

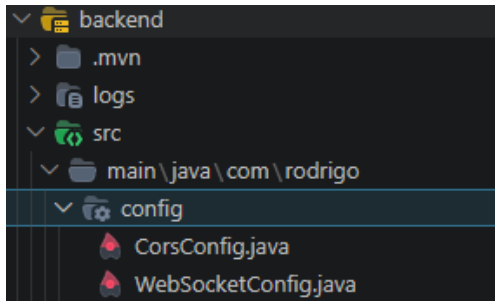


Por último, en esta captura, los dos móviles funcionando muestran datos distintos de grupos, esto se debe a que el usuario de la derecha, se encuentra en el “Grupo Prueba 1”, por lo que para su interfaz no es necesario mostrar su propio grupo, por el otro lado, el usuario de la izquierda sí que puede ver el “Grupo Prueba 1”, ya que no se encuentra en ese grupo, por lo que es una opción para unirse

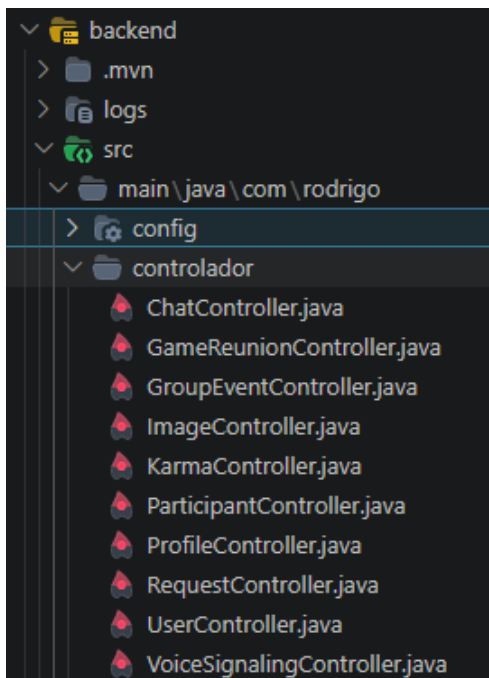
Anexo 5 - Estructura del proyecto

Backend

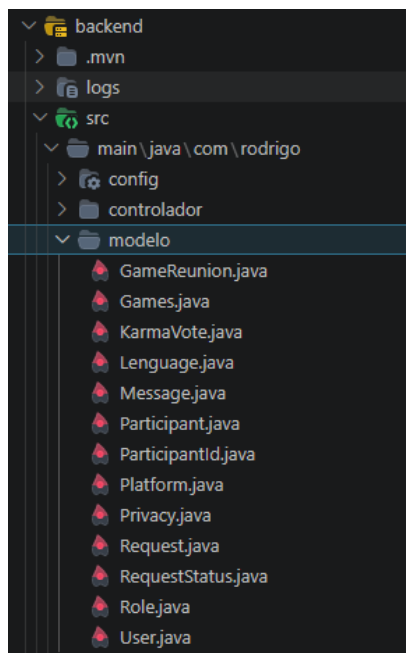
Config



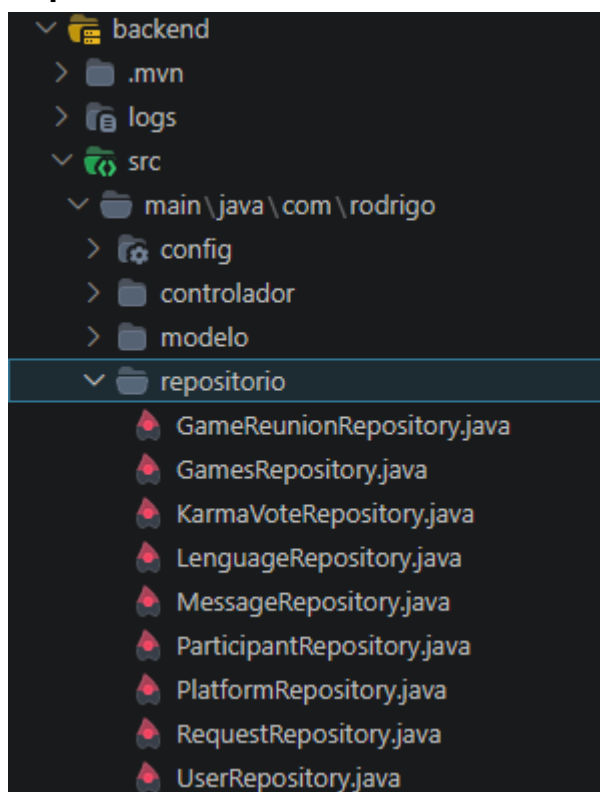
Controlador



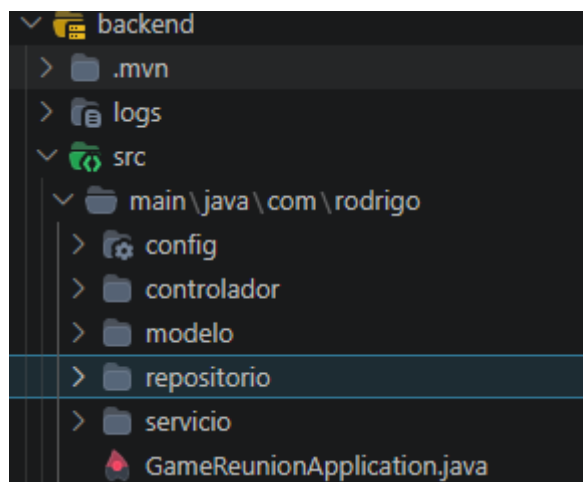
Modelo



Repositorio

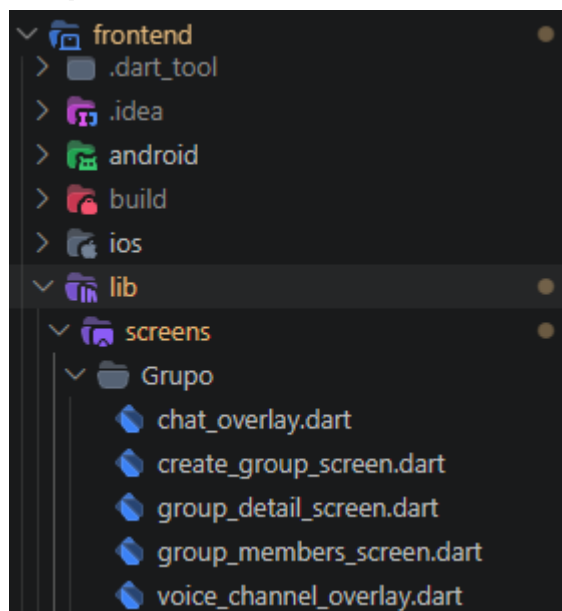


Servicio

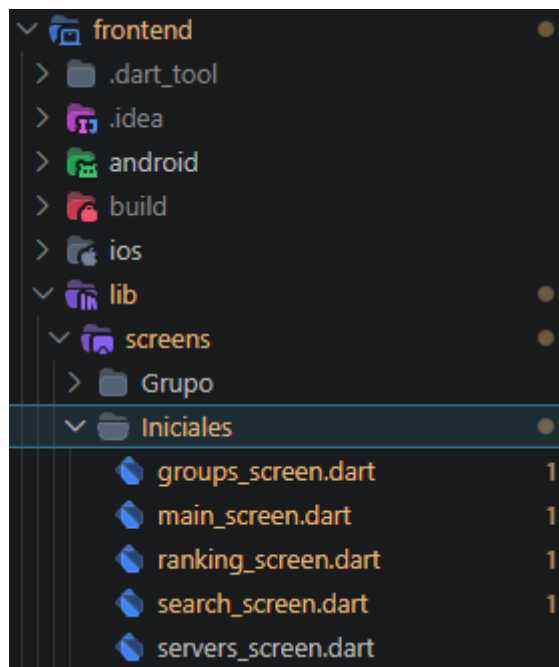


Frontend

Grupo



Iniciales



Registro

